Programação para Dispositivos Móveis

Atividade 20.2 – Persistência com SqFlite

App de Contatos (armazenar, alterar, consultar e excluir)

Tabela contatos: id (INTEGER PRIMARY KEY AUTOINCREMENT), nome, email

Pacotes: sqflite, path

1. pubspec.yaml (dependências)

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  sqflite: ^2.4.5
  path: ^1.9.1
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
  sqflite_common_ffi: ^2.4.5
```

2. lib/contato.dart (modelo)

```
class Contato {
  const Contato({this.id, required this.nome, required this.email});

  /// Nulo enquanto o contato ainda não foi salvo no banco.
  final int? id;
  final String nome;
  final String email;

  Map<String, Object?> toMap() => {'id': id, 'nome': nome, 'email': email};

  factory Contato.fromMap(Map<String, Object?> map) => Contato(
        id: map['id'] as int,
        nome: map['nome'] as String,
        email: map['email'] as String,
      );
}
```

3. lib/banco_contatos.dart (acesso ao SqFlite)

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

import 'contato.dart';

/// Acesso à tabela `contatos` (id, nome, email) com SqFlite.
class BancoContatos {
  /// [caminho] permite usar outro arquivo (ex.: banco em memória nos testes).
  BancoContatos({this.caminho});

  final String? caminho;

  // Abre o banco uma única vez, na primeira operação.
  late final Future<Database> _banco = _abrirBanco();

  Future<Database> _abrirBanco() async {
    final caminho = this.caminho ?? join(await getDatabasesPath(), 'banco.db');
    return openDatabase(
      caminho,
      version: 1,
      onCreate: (db, versaoRecente) async {
        await db.execute(
          'CREATE TABLE contatos ('
          'id INTEGER PRIMARY KEY AUTOINCREMENT, '
          'nome VARCHAR NOT NULL, '
          'email VARCHAR NOT NULL)',
        );
      },
    );
  }

  /// Armazena um novo contato e retorna o id gerado.
  Future<int> salvar(Contato contato) async {
    final db = await _banco;
    return db.insert('contatos', {'nome': contato.nome, 'email': contato.email});
  }

  /// Consulta os contatos em ordem alfabética, filtrando por nome ou email.
  Future<List<Contato>> consultar({String filtro = ''}) async {
    final db = await _banco;
    final termo = filtro.trim();
    final linhas = await db.query(
      'contatos',
      where: termo.isEmpty ? null : 'nome LIKE ? OR email LIKE ?',
      whereArgs: termo.isEmpty ? null : ['%$termo%', '%$termo%'],
      orderBy: 'nome COLLATE NOCASE',
    );
    return linhas.map(Contato.fromMap).toList();
  }

  /// Altera nome e email do contato com o mesmo id.
  Future<int> alterar(Contato contato) async {
    final db = await _banco;
    return db.update(
      'contatos',
      {'nome': contato.nome, 'email': contato.email},
      where: 'id = ?',
      whereArgs: [contato.id],
    );
  }

  /// Exclui o contato com o id informado.
```

```
  Future<int> excluir(int id) async {
    final db = await _banco;
    return db.delete('contatos', where: 'id = ?', whereArgs: [id]);
  }

  Future<void> fechar() async => (await _banco).close();
}
```

4. lib/main.dart (telas)

```
import 'package:flutter/material.dart';

import 'banco_contatos.dart';
import 'contato.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Contatos',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
      ),
      home: const ContatosPage(),
    );
  }
}

class ContatosPage extends StatefulWidget {
  const ContatosPage({super.key});

  @override
  State<ContatosPage> createState() => _ContatosPageState();
}

class _ContatosPageState extends State<ContatosPage> {
  final BancoContatos _banco = BancoContatos();
  final TextEditingController _busca = TextEditingController();
  List<Contato> _contatos = [];

  @override
  void initState() {
    super.initState();
    _consultar();
  }

  @override
  void dispose() {
    _busca.dispose();
    _banco.fechar();
    super.dispose();
  }

  Future<void> _consultar() async {
    final contatos = await _banco.consultar(filtro: _busca.text);
    if (!mounted) return;
    setState(() => _contatos = contatos);
  }

  /// Abre o formulário vazio (novo contato) ou preenchido (alteração).
  Future<void> _abrirFormulario([Contato? contato]) async {
    final resultado = await showDialog<Contato>(
      context: context,
      builder: (context) => FormularioContato(contato: contato),
    );
```

```
  if (resultado == null) return;

  if (resultado.id == null) {
    await _banco.salvar(resultado);
    _mostrarMensagem('Contato salvo');
  } else {
    await _banco.alterar(resultado);
    _mostrarMensagem('Contato alterado');
  }
  await _consultar();
}

Future<void> _excluir(Contato contato) async {
  final confirmar = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Excluir contato'),
      content: Text('Deseja excluir "${contato.nome}"?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Cancelar'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Excluir'),
        ),
      ],
    ),
  );
  if (confirmar != true) return;

  await _banco.excluir(contato.id!);
  _mostrarMensagem('Contato excluído');
  await _consultar();
}

void _mostrarMensagem(String texto) {
  if (!mounted) return;
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text(texto)));
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      title: const Text('Contatos'),
    ),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(12),
          child: TextField(
            controller: _busca,
            decoration: const InputDecoration(
              labelText: 'Consultar por nome ou email',
              prefixIcon: Icon(Icons.search),
              border: OutlineInputBorder(),
            ),
            onChanged: (_) => _consultar(),
          ),
```

```
          ),
          Expanded(
            child: _contatos.isEmpty
                ? const Center(child: Text('Nenhum contato encontrado'))
                : ListView.separated(
                    itemCount: _contatos.length,
                    separatorBuilder: (context, index) =>
                        const Divider(height: 1),
                    itemBuilder: (context, index) {
                      final contato = _contatos[index];
                      return ListTile(
                        leading: CircleAvatar(child: Text('${contato.id}')),
                        title: Text(contato.nome),
                        subtitle: Text(contato.email),
                        onTap: () => _abrirFormulario(contato),
                        trailing: IconButton(
                          icon: const Icon(Icons.delete_outline),
                          tooltip: 'Excluir',
                          onPressed: () => _excluir(contato),
                        ),
                      );
                    },
                  ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _abrirFormulario,
        tooltip: 'Novo contato',
        child: const Icon(Icons.add),
      ),
    );
  }
}

class FormularioContato extends StatefulWidget {
  const FormularioContato({super.key, this.contato});

  final Contato? contato;

  @override
  State<FormularioContato> createState() => _FormularioContatoState();
}

class _FormularioContatoState extends State<FormularioContato> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _nome =
      TextEditingController(text: widget.contato?.nome);
  late final TextEditingController _email =
      TextEditingController(text: widget.contato?.email);

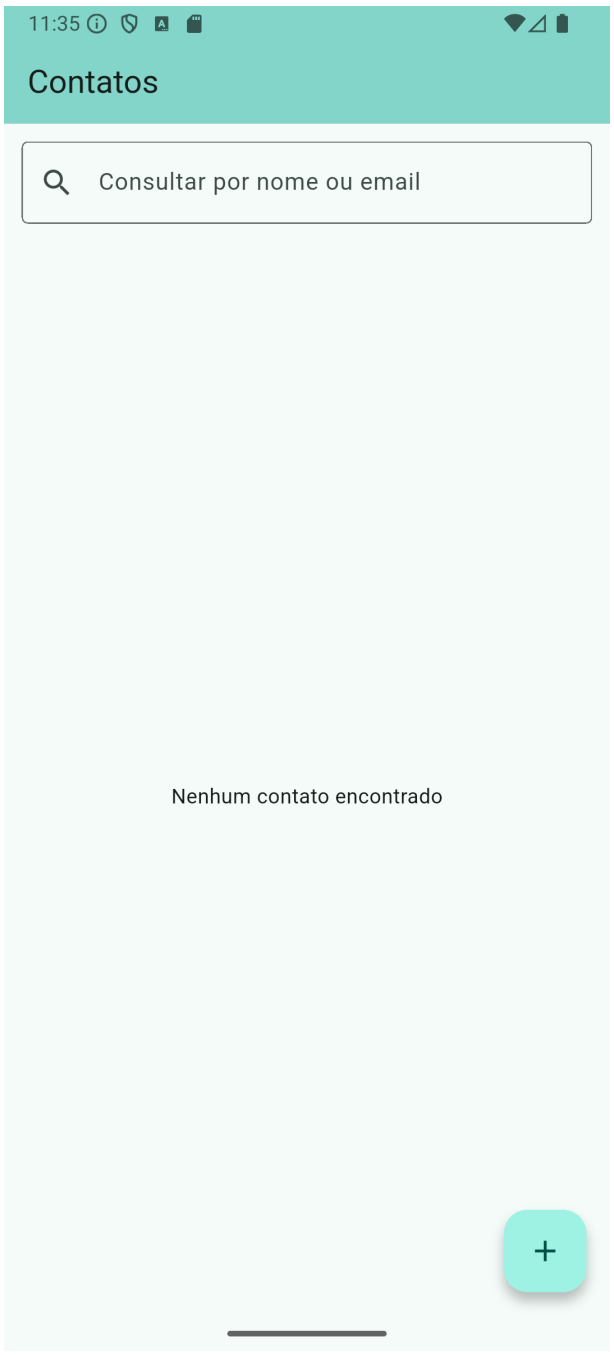
  @override
  void dispose() {
    _nome.dispose();
    _email.dispose();
    super.dispose();
  }

  void _confirmar() {
    if (!_formKey.currentState!.validate()) return;
    Navigator.pop(
      context,
      Contato(
        id: widget.contato?.id,
```

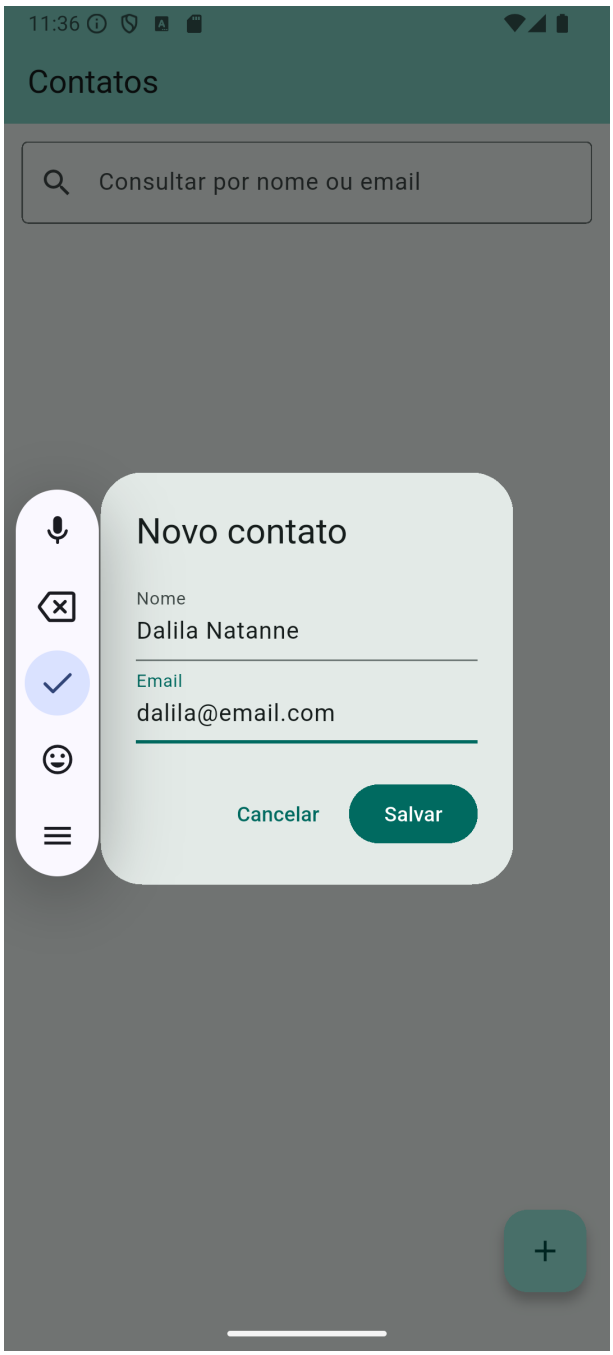
```
        nome: _nome.text.trim(),
        email: _email.text.trim(),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text(widget.contato == null ? 'Novo contato' : 'Alterar contato'),
      content: Form(
        key: _formKey,
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            TextFormField(
              controller: _nome,
              decoration: const InputDecoration(labelText: 'Nome'),
              textCapitalization: TextCapitalization.words,
              validator: (valor) =>
                  (valor == null || valor.trim().isEmpty) ? 'Informe o nome' : null,
            ),
            TextFormField(
              controller: _email,
              decoration: const InputDecoration(labelText: 'Email'),
              keyboardType: TextInputType.emailAddress,
              validator: (valor) {
                final email = valor?.trim() ?? '';
                if (email.isEmpty) return 'Informe o email';
                if (!RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$').hasMatch(email)) {
                  return 'Email inválido';
                }
                return null;
              },
              onFieldSubmitted: (_) => _confirmar(),
            ),
          ],
        ),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Cancelar'),
        ),
        FilledButton(onPressed: _confirmar, child: const Text('Salvar')),
      ],
    );
  }
}
```

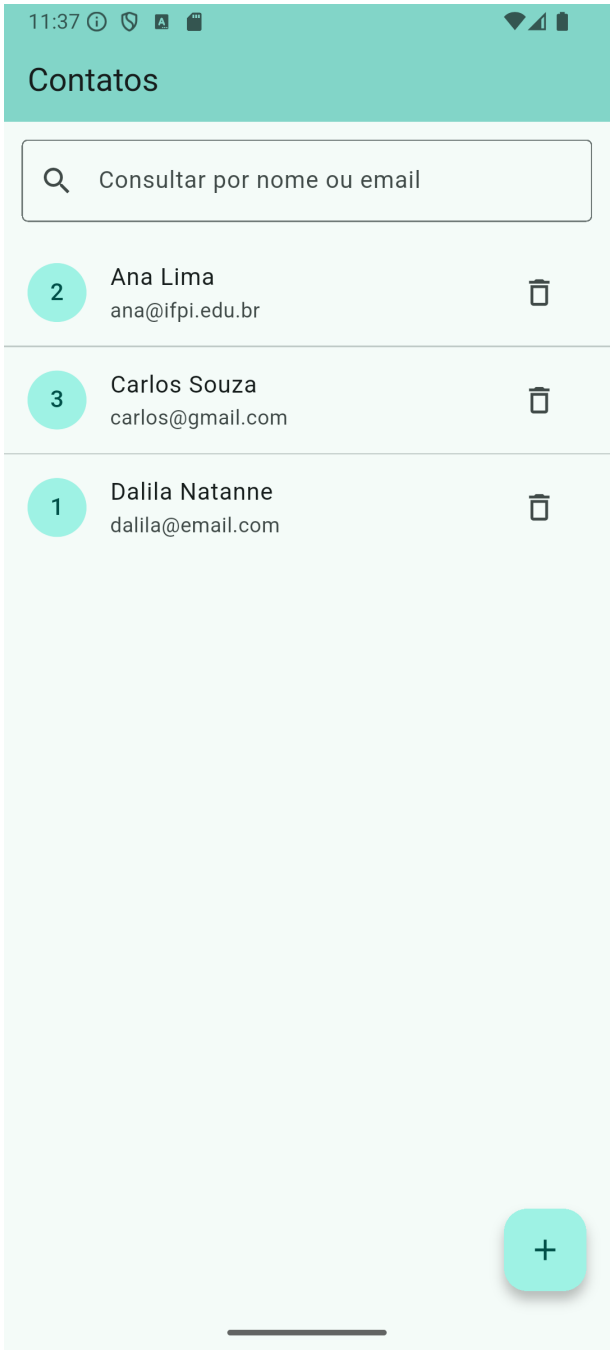
5. Prints das telas (emulador Android)



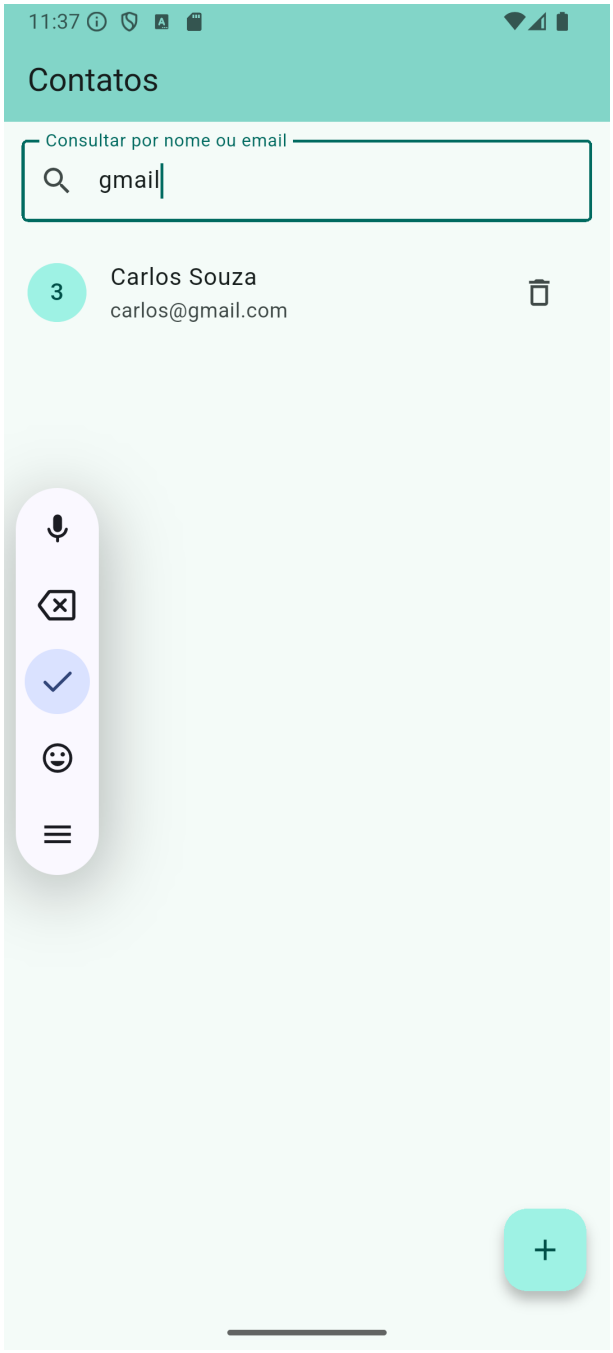
1. Tela inicial (banco vazio)



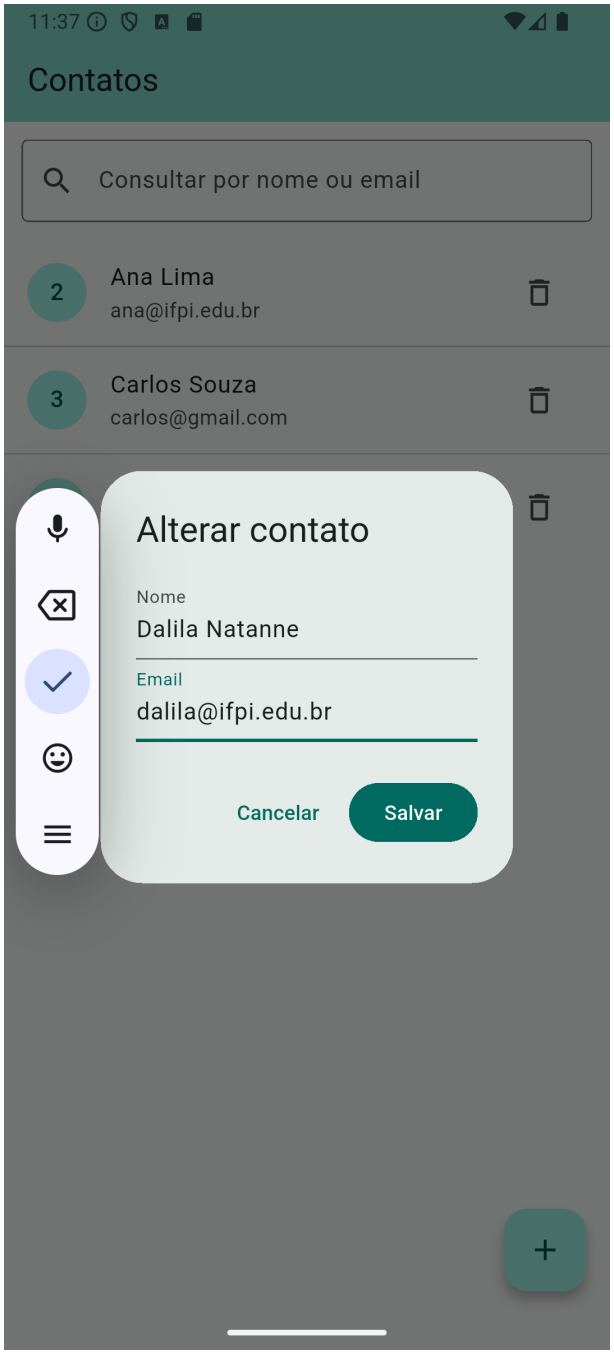
2. Armazenar: novo contato



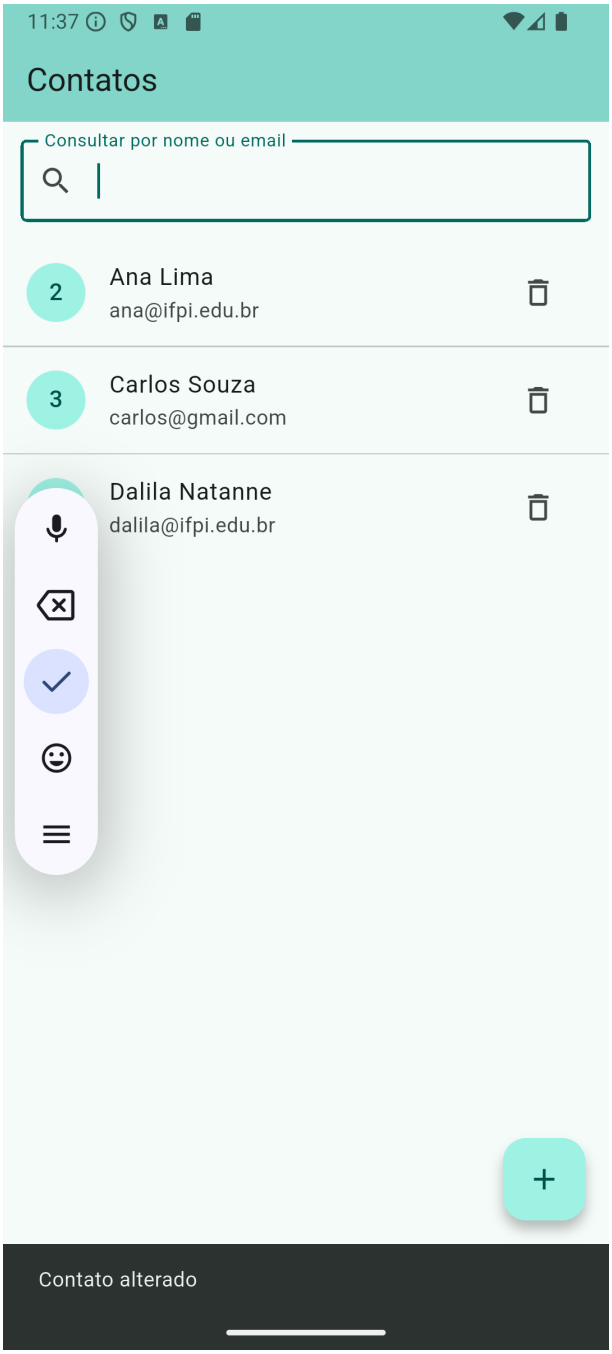
3. Contatos salvos (ordem alfabética)



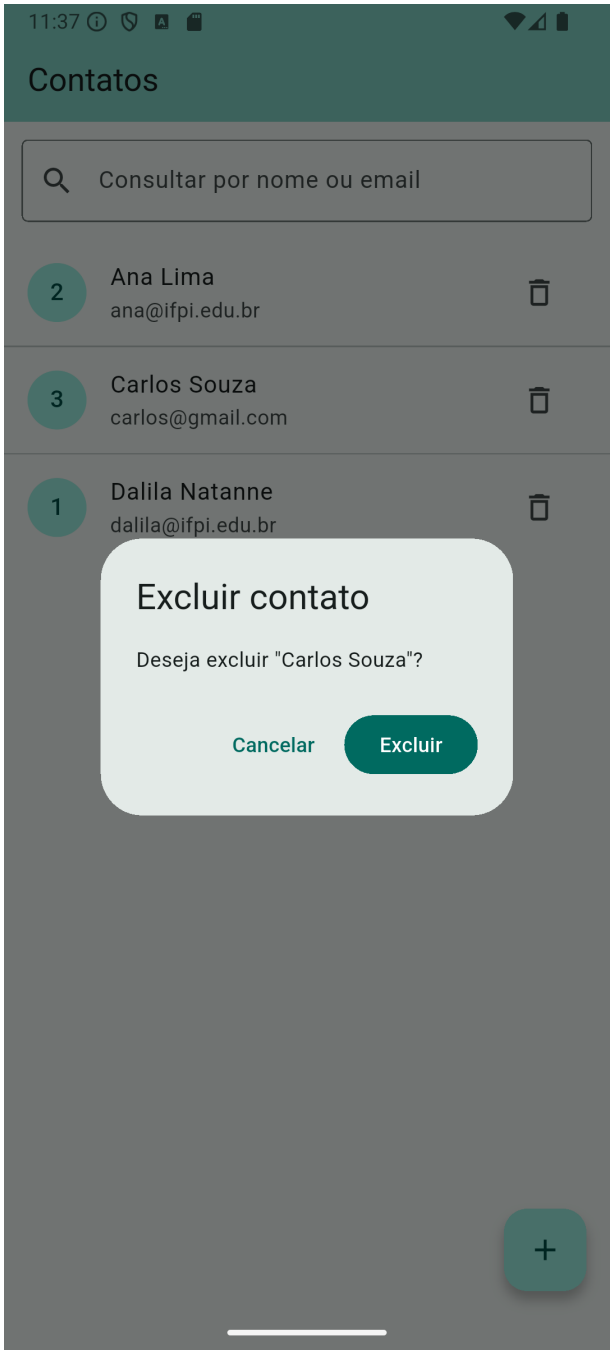
4. Consultar: filtro "gmail"



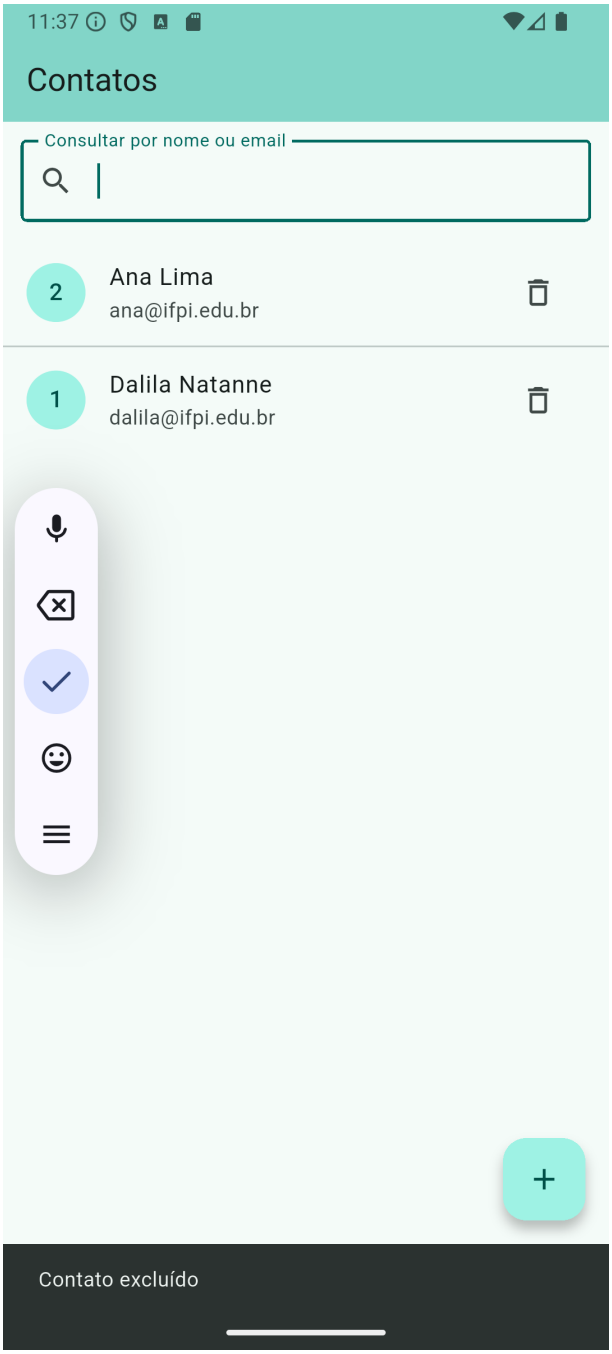
5. Alterar: email modificado



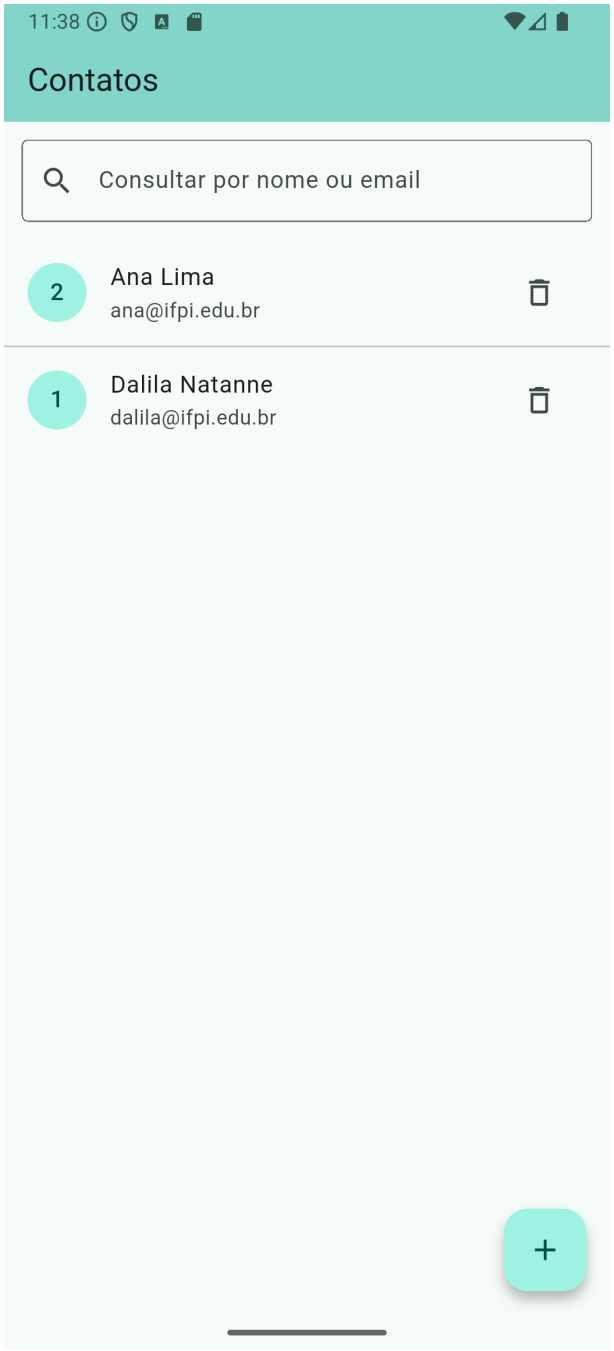
6. Contato alterado



7. Excluir: confirmação



8. Contato excluído



9. Dados persistidos após fechar e reabrir o app